

FUNDAMENTOS DE PROGRAMAÇÃO PARA INTERNET



Prof. Tiago Piperno Bonetti
bonetti@prof.unipar.br

JavaScript

Um dos muitos métodos HTML JavaScript é ***getElementById()***.

O exemplo abaixo "encontra" um elemento HTML (com id = "demo") e altera o conteúdo do elemento (innerHTML) para "Hello JavaScript":

```
<p id="demo">JavaScript pode alterar o conteúdo HTML.</p>
```

```
<button type="button"  
onclick='document.getElementById("demo").innerHTML = "Hello JavaScript!"'>  
Click Me! </button>
```

JavaScript

Neste exemplo, o JavaScript altera o valor do src de uma tag :

```
<button onclick="document.getElementById('lampada').src='pic_bulbon.gif'">  
Ligue a lâmpada</button>
```

```

```

```
<button onclick="document.getElementById('lampada').src='pic_bulboff.gif'">  
Desligue a lâmpada</button>
```

JavaScript

Ocultar elementos HTML pode ser feito alterando o *display* estilo:

```
<p id="texto">JavaScript pode esconder elementos HTML .</p>
```

```
<button type="button"  
onclick="document.getElementById('texto').style.display='none'">  
Click aqui para ocultar</button>
```

JavaScript

Mostrar elementos HTML pode ser feito alterando o *display* estilo:

```
<p id="texto2" style="display:none;">
```

JavaScript pode mostrar elementos HTML.

```
</p>
```

```
<button type="button"
```

```
onclick="document.getElementById('texto2').style.display='block'">
```

```
Click aqui para mostrar</button>
```

JavaScript - Output

Possibilidades de exibição de JavaScript

JavaScript pode "exibir" dados de maneiras diferentes:

- Escrevendo em um elemento HTML, usando `innerHTML`.
- Escrevendo na saída HTML usando `document.write()`.
- Escrevendo em uma caixa de alerta, usando `window.alert()`.
- Escrevendo no console do navegador, usando `console.log()`.

JavaScript - Output

Usando innerHTML

Para acessar um elemento HTML, JavaScript pode usar o método:
`document.getElementById(id)`.

O atributo *id* define o elemento HTML. A propriedade innerHTML define o conteúdo HTML

```
<p id="soma"></p>
```

```
<script>document.getElementById("soma").innerHTML = 5 + 6; </script>
```

JavaScript - Output

Usando document.write ()

Para fins de teste, é conveniente usar **document.write()**:

```
<h1>Texto comum</h1>
```

```
<script>  
    document.write('Texto JS: ', 5 + 6);  
</script>
```


JavaScript - Output

Usando *window.alert()*

Você pode usar uma caixa de alerta para exibir dados:

```
<script>  
    window.alert(5 + 6);  
</script>
```

JavaScript - Output

Usando ***console.log ()***

Para fins de depuração, você pode chamar o método `console.log()` no navegador para exibir dados.

```
<script>  
    console.log(5 + 6);  
</script>
```

JavaScript - Output

Você pode chamar o método ***window.print()*** no navegador para imprimir o conteúdo da janela atual.

```
<button onclick="window.print()">  
    Imprima essa página  
</button>
```

JavaScript - Variáveis

As **regras** gerais para construir nomes para variáveis (identificadores únicos) são:

- ✓ Os nomes podem conter letras, dígitos, sublinhados e cifrões.
- ✓ Os nomes devem começar com uma letra.
- ✓ Os nomes também podem começar com \$ e _.
- ✓ Os nomes diferenciam maiúsculas de minúsculas (y e Y são variáveis diferentes).
- ✓ Palavras reservadas (como palavras-chave JavaScript) não podem ser usadas como nomes.

JavaScript - Variáveis

No exemplo abaixo, criamos uma variável chamada *carName* e atribuímos o valor "Volvo" a ela.

Em seguida, "exibimos" o valor dentro de um parágrafo HTML com id = "carro":

```
<p id="carro"></p>
```

```
<script>
```

```
    var carName = "Volvo";
```

```
    document.getElementById("carro").innerHTML = carName;
```

```
</script>
```

JavaScript - Variáveis

Os **objetos** JavaScript são escritos com chaves { }.

As propriedades do objeto são escritas como **nome: valor**, separados por vírgulas.

```
var person = {firstName:"John", lastName:"Doe", age:50, eyeColor:"blue"};
```

O objeto (pessoa) no exemplo acima tem 4 propriedades: firstName, lastName, age e eyeColor.

Operadores aritméticos

Operador	Descrição
+	Adição
-	Subtração
*	Multiplicação
**	Exponenciação
/	Divisão
%	Módulo
++	Incremento
--	Decremento

Operadores de atribuição

Operador	Exemplo	Igual a
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y
**=	x **= y	x = x ** y

Operadores lógicos

Operador	Descrição
&&	lógico e
 	lógico ou
!	lógico não

Funções

Uma função JavaScript é um **bloco de código** projetado para executar uma **tarefa específica**.

Uma função JavaScript é executada quando "algo" a **invoca** (chama).

```
function nomeFuncao(p1, p2) { //criando a função
    return p1 * p2;    // retorno da função (multiplica os dois números)
}
```

Funções

Uma função JavaScript é definida com a palavra ***function***, seguida por um ***nome***, seguido por ***parênteses*** ().

Os nomes das funções podem conter letras, dígitos, sublinhados e cifrões (mesmas regras das variáveis).

Os parênteses podem incluir nomes de ***parâmetros*** separados por vírgulas.

O ***código*** a ser executado, pela função, é colocado entre chaves: { }

```
function nome(parametro 1, parametro 2) {  
    //código  
}
```

Invocação de Função

O código dentro da função será executado quando "algo" invocar (chamar) a função:

- ✓ Quando ocorre um evento (quando um usuário clica em um botão).
- ✓ Quando é invocado (chamado) a partir do código JavaScript.
- ✓ Automaticamente (auto-invocado).

Retorno de Função

Quando o JavaScript atinge uma instrução **return**, a função para de ser executada.

Se a função foi chamada a partir de uma **instrução**, o JavaScript "retornará" para executar o código após a instrução de chamada.

As funções geralmente calculam um valor de retorno. O valor de retorno é "retornado" de volta ao "chamador":

```
var x = minhaFuncao(4, 3); // A função é chamada, o valor de retorno terminará em x
function minhaFuncao(num1, num2) {
    return num1 * num2; // A função retorna o produto de num1 e num2
}
```

O operador () invoca a função

```
function toCelsius(fahrenheit) {  
    return (5/9) * (fahrenheit-32);  
}  
document.getElementById("demo").innerHTML = toCelsius(42);
```

Funções usadas como valores variáveis

//Usar uma variável para armazenar o valor de retorno:

```
var x = toCelsius(77);
```

```
var text = "A temperatura é " + x + " Celsius";
```

//Você pode usar a função diretamente, como um valor de variável:

```
var text = " A temperatura é " + toCelsius(77) + " Celsius";
```

Eventos HTML

Um evento HTML pode ser algo que o navegador faz ou um usuário.

Aqui estão alguns exemplos de eventos HTML:

- ✓ Uma página HTML da web terminou de carregar.
- ✓ Um campo de entrada HTML foi alterado.
- ✓ Um botão HTML foi clicado.

JavaScript permite executar código quando eventos são detectados.

HTML permite que atributos de manipulador de eventos, com código JavaScript, sejam adicionados a elementos HTML.

Eventos HTML

<!-- Com aspas simples: -->

<element event='JavaScript'></element>

<!-- Com aspas duplas: -->

<element event="JavaScript"></element>

<p id="horario"></p>

<button onclick="document.getElementById('horario').innerHTML = Date()">

Que horas são?</button>

Eventos HTML

<!-- o código altera o conteúdo de seu próprio elemento
(usando this.innerHTML): -->

<button onclick="this.innerHTML = Date()">Que horas são?</button>

Eventos HTML

O código JavaScript geralmente tem várias linhas. É mais comum ver atributos de evento chamando funções:

```
<button onclick="displayDate()">Que horas são?</button>
<script>
    function displayDate() {
        document.getElementById("horario2").innerHTML = Date();
    }
</script>
<p id="horario2"></p>
```

Declarações Condicionais

Use a *if* para especificar um bloco de código JavaScript a ser executado se uma condição for verdadeira.

```
if (condição) {  
    // bloco de código  
}
```

Declarações Condicionais

```
<p id="saudacao">Boa noite!</p>
```

```
<script>
```

```
    //Faça uma saudação de "Bom dia" se a hora for inferior a 18:00:
```

```
    if (new Date().getHours() < 18) {
```

```
        document.getElementById("saudacao").innerHTML = "Bom dia!";
```

```
    }
```

```
</script>
```

Declarações Condicionais

Use a **else** para especificar um bloco de código a ser executado se a condição for falsa.

```
<p id="saudacao2"></p>
<script>
    var saudacao;
    var hora = new Date().getHours();

    if (hora < 18) {
        saudacao = "Bom dia!";
    } else {
        saudacao = "Boa noite!";
    }
    document.getElementById("saudacao2").innerHTML = saudacao;
</script>
```

Declarações Condicionais

Use a **else if** para especificar uma nova condição se a primeira condição for falsa.

```
<p id="saudacao3"></p>
<script>
    var saudacao;
    var hora = new Date().getHours();

    if (hora < 12) {
        saudacao = "Bom dia!";
    } else if (hora < 19) {
        saudacao = "Boa tarde!";
    } else {
        saudacao = "Boa noite!";
    }
    document.getElementById("saudacao3").innerHTML = saudacao;
</script>
```

Referências

W3schools. <https://www.w3schools.com/js/default.asp>

Caelum. <https://www.caelum.com.br/apostila-html-css-javascript/>

Developer Mozilla:

[https://developer.mozilla.org/pt-](https://developer.mozilla.org/pt-BR/docs/Aprender/Getting_started_with_the_web/JavaScript_basico)

[BR/docs/Aprender/Getting_started_with_the_web/JavaScript_basico](https://developer.mozilla.org/pt-BR/docs/Aprender/Getting_started_with_the_web/JavaScript_basico)